Practical 1: Password Strength Checker

Aim

To check password strength using regular expressions and classify it as Weak, Medium, or Strong.

Algorithm

1. Check minimum length (>= 8).
2. Check for uppercase letters.
3. Check for digits.
4. Check for special characters.
5. Count satisfied rules and display strength.

Program Code **PasswordChecker.java**

```
import java.util.ArrayList; import
java.util.List;
public class PasswordChecker
{

    public static boolean hasMinLength(String pw) {
return pw != null && pw.length() >= 8;
    }
    public static boolean hasUppercase(String pw) {
return pw != null && pw.matches(".*[A-Z].*");
    }
    public static boolean hasDigit(String pw) {
return pw != null && pw.matches(".*[0-9].*");
    }     public static boolean hasSpecialChar(String pw) {
// Matches any non-alphanumeric character         return pw
!= null && pw.matches(".*[^a-zA-Z0-9].*");
    }
    /**
     * Evaluates the password, prints rule statuses, and returns strength
label.      */     public static String evaluatePassword(String pw) {
List<String> passedRules = new ArrayList<>();
        List<String> failedRules = new ArrayList<>();

        if (hasMinLength(pw)) passedRules.add("Length ≥ 8");
else failedRules.add("Length ≥ 8");

        if (hasUppercase(pw)) passedRules.add("Uppercase letter");
else failedRules.add("Uppercase letter");
        if (hasDigit(pw)) passedRules.add("Digit");
else failedRules.add("Digit");
```

```
        if (hasSpecialChar(pw)) passedRules.add("Special character");
else failedRules.add("Special character");
        int score =
passedRules.size();         String
label = strength(score);

        System.out.println("Testing password: \"" + pw + "\"");
System.out.println("  Passed (" + score + "/4): " + passedRules);         if
(!failedRules.isEmpty()) {
            System.out.println("  Failed: " + failedRules);
        }
        System.out.println("  Result: " + label + "\n");
        return
label;
    }
    public static String strength(int score) {
if (score <= 1) {             return "Weak";
} else if (score <= 3) {             return
"Medium";
        } else {
            return "Strong";
        }
    }
}
```

Driver1.java

```
public class Driver1 {     public static
void main(String[] args) {
        String[] testPasswords = {
                "abc",          // Weak (0 rules passed)
                "password123",  // Medium (2 rules passed: length, digit)
                "Abcd1234!",    // Strong (4 rules passed)
                "JAVA"          // Weak (1 rule passed: uppercase)
        };
        for (String pw : testPasswords) {
PasswordChecker.evaluatePassword(pw);
        }
    }
}
```

Sample Output abc

→ Weak

Abcd1234! → Strong

Result

The program was compiled and executed successfully, and the desired output was obtained.

Practical 2: Chat Log Filter

Aim

To filter chat log messages using string manipulation and keyword searching.

Algorithm

6. Store log lines in an array.
7. Read a keyword from the user.
8. Split each line into time, user, and message.
9. Ignore malformed lines.
10. Display matching messages and count.

Program Code

ChatFilter.java

```
public class ChatFilter {
    /**
     * Filters log lines based on keyword search inside the message part.
     */
    public static String filterLogs(String[] logs, String keyword) {
int matchCount = 0;
        StringBuilder report = new StringBuilder();
        String lowerKeyword = keyword.toLowerCase();
         for (String line : logs)
{
            // Split into at most 3 parts: [time, user, message]
String[] parts = line.split(" ", 3);

            // Skip malformed lines with fewer than 3 parts
if (parts.length < 3) {                continue;
            }

            String time = parts[0];
            String user = parts[1];
            String message = parts[2];
            // Case-insensitive check
            if (message.toLowerCase().contains(lowerKeyword)) {
matchCount++;                report.append(time)
.append(" ")
                        .append(user)
                        .append(": ")
                        .append(message)
                        .append("\n");
            }
        }         return "Matches: " + matchCount + "\n" +
report.toString();     }
}
```

Driver2.java

```
import java.util.Scanner;
```

```
public class Driver2 {
    public static void main(String[] args) {
        // Sample log lines (including one malformed line)
        String[] logs = {
                "10:05 alice Hello there",
                "10:06 bob corrupt line",
                "10:07 charlie HELLO world!",
                "10:08 System" // Malformed line (< 3 parts)
        };

        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter keyword to filter: ");
        String keyword = scanner.nextLine();

        String result = ChatFilter.filterLogs(logs, keyword);
        System.out.println("\n--- Filter Results ---");
        System.out.print(result);

scanner.close();
    }
}
```

Sample Output

Keyword: hello

Matches: 2

Result

The program was compiled and executed successfully, and the desired output was obtained.

Practical 3: Template Filler (Advanced Learners)

Aim

To replace placeholders in a template string using Pattern and Matcher classes.

Algorithm

11. Create a template with placeholders.
12. Use Pattern and Matcher.
13. Replace placeholders using parallel arrays.
14. Replace missing values with [?].
15. Display the final template.

Program Code

TemplateFiller.java

```
import java.util.regex.Matcher; import
java.util.regex.Pattern;
```

```
public class TemplateFiller
{
    /**
* Replaces placeholders in the format {key} using key-value parallel arrays.
* Unmatched placeholders are replaced with "[?]".
     */
    public static String fill(String template, String[] names, String[] values)
{
        // Pattern matches {placeholder} and captures the key name inside group
1
        Pattern pattern = Pattern.compile("\\{(\\w+)\\}");
        Matcher matcher = pattern.matcher(template);
        StringBuilder sb = new StringBuilder();
         while (matcher.find())
{
            String key = matcher.group(1); // Extracted key inside braces, e.g.
"name"
            String replacement = lookupValue(key, names, values);

            // Matcher.quoteReplacement prevents special character ($ / \)
misinterpretation            matcher.appendReplacement(sb,
Matcher.quoteReplacement(replacement));
        }
        matcher.appendTail(sb);
         return
sb.toString();
    }
    /**
     * Looks up the key in the names array and returns the corresponding value,
* or "[?]" if the key is not found.
     */
    private static String lookupValue(String key, String[] names, String[]
values) {         for (int i = 0; i < names.length; i++) {             if
(names[i].equalsIgnoreCase(key)) {
                return values[i];
            }
}
        return "[?]";
    }
}
```

Driver3.java

```
public class Driver3 {
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";

        // Parallel arrays (key-value mapping)
        String[] names = {"name", "id"};
        String[] values = {"Riya", "A07"};

        String filledTemplate = TemplateFiller.fill(template, names, values);

        System.out.println("Original Template: " + template);
```

```
        System.out.println("Filled Result:     " + filledTemplate);
}
}
```

Sample Output

Dear Riya, order A07 ships [?].

Result

The program was compiled and executed successfully, and the desired output was obtained.